

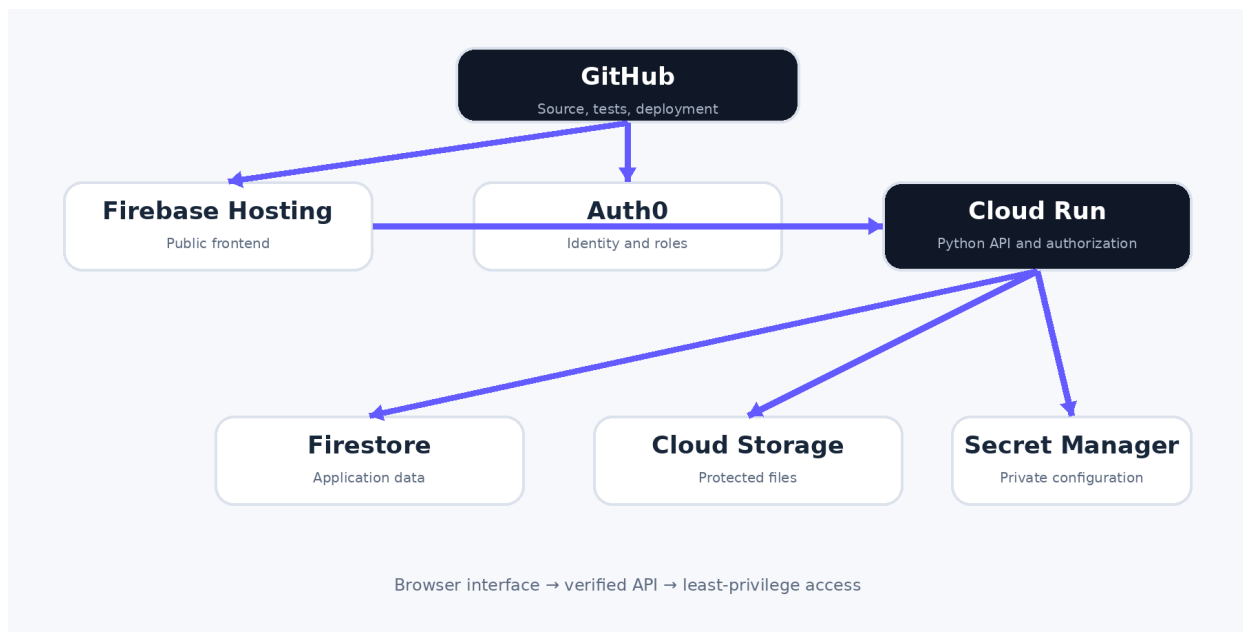
LESSON 6

From Static Website to Secure, Scalable Web Application

The new architecture for swe.tickerbell.biz — chosen for learning today and scalability tomorrow

THE DECISION

Keep GitHub as the source of truth. Move the frontend to Firebase Hosting, run the Python API on Cloud Run, store application data in Firestore, keep protected files in private Cloud Storage, use Auth0 for identity, and connect deployments through GitHub Actions.



Architecture at a glance

Learning objectives

By the end of this lesson, you should be able to:

- Explain why a public static site cannot securely hold private learner or admin data.
- Distinguish authentication from authorization.
- Name the responsibility of Firebase Hosting, Auth0, Cloud Run, Firestore, Cloud Storage, Secret Manager, and GitHub Actions.
- Describe the protected request flow from browser login to stored data.
- Apply a three-role model without building the entire platform at once.
- Identify which information belongs in public and private repositories.

1. Why we are changing the architecture

The existing GitHub Pages website is not a mistake. It is the correct architecture for a public learning journal made from HTML, CSS, JavaScript, images, and downloadable public documents. It is inexpensive, transparent, and easy to publish.

The product requirements have now changed. We want learner accounts, team content management, admin analytics, saved progress, protected files, and auditable actions. A browser-only site cannot securely provide those capabilities because every published file can be requested directly and all browser code is visible to visitors.

CORE PRINCIPLE

Change architecture when requirements change. The purpose of this redesign is not to collect fashionable services; it is to learn each layer by solving a real product need.

2. Website versus web application

Public static website	Authenticated web application
Publishes the same files to every visitor	Returns information according to identity, ownership, and permission
Runs presentation logic in the browser	Runs security and business rules on a trusted backend
Public documents can be downloaded directly	Private files require validated, time-limited access
No persistent learner state	Profiles, progress, content status, and audit records are stored
Excellent for the journal and previews	Required for learner, team, and admin operations

3. The selected architecture

Responsibility	Selected service	Why it belongs here
Source and workflow	GitHub	Version control, review, tests, and deployment automation
Public frontend	Firebase Hosting	Static hosting, HTTPS, custom domain, previews, Google integration
Identity	Auth0	Universal Login, user identity, roles, and API access tokens
Trusted backend	Cloud Run + FastAPI	Token validation, authorization, business logic, and API endpoints
Application data	Firestore	Serverless document data for profiles, lessons, progress, and logs
Protected files	Private Cloud Storage	PDFs, uploads, and generated files outside public GitHub
Secrets	Secret Manager	Backend configuration that must never enter source code
Monitoring	Cloud Logging + analytics	Operational errors, service health, and aggregated product insight

Why Firebase Hosting rather than Cloudflare Pages?

Both are strong static hosting platforms. Firebase Hosting is the better fit for this learning project because the chosen backend, database, storage, monitoring, and billing already live in Google Cloud. This reduces cross-platform configuration while preserving GitHub as the central development workflow.

Why Firestore rather than Cloud SQL now?

Firestore removes the need to run a database server continuously and fits documents such as learner profiles, lesson records, progress, content items, and audit entries. PostgreSQL remains a future option when complex relationships, financial records, or advanced reporting justify a relational database.

4. How a protected request works

1. The visitor opens the frontend served by Firebase Hosting.
2. The visitor selects Log in and completes Auth0 Universal Login.
3. Auth0 returns an access token intended for the Become SWE API.
4. The browser sends the token in the Authorization header when calling Cloud Run.
5. The FastAPI backend verifies the signature, issuer, audience, expiry, and permissions.
6. The backend checks the user role and, where relevant, ownership of the requested record.
7. The Cloud Run service account accesses Firestore or private Cloud Storage with limited Google IAM permissions.
8. The API returns only the authorized result, and important admin actions are written to an audit log.

DO NOT CONFUSE THE TWO

Authentication answers “Who is this user?” Authorization answers “What may this user do with this specific resource?” Successful login alone is never enough to authorize an admin operation.

5. The three access levels

We keep the complete learner, team, and admin design, but the first release builds only one meaningful page for each role. This keeps the authentication exercise small while demonstrating real role separation.

Role	First protected page	First responsibility	Initial permission
Learner	/learn/dashboard	Continue lessons and see personal progress	read:lessons
Team	/team/content	Draft and manage lesson content	manage:content
Admin	/admin/analytics	View safe, aggregated platform metrics	view:analytics

Role inheritance should remain explicit

The team and admin interfaces may include learner capabilities, but permissions should be evaluated explicitly by the API. Avoid a loose rule such as “admin can do everything” until every powerful operation has been identified, tested, and audited.

6. Repository architecture

Repository	Visibility	Contents	Never store
Become-SWE	Public	Lessons, Word/PDF files, tutorials, decisions, mistakes, learning record	Private files, customer data, secrets
swe-tickerbell	Public	Frontend HTML, CSS, JavaScript, images, safe public configuration	Client secrets, Management API tokens, protected PDFs
swe-backend	Private	FastAPI code, authorization, data models, tests, deployment definitions	Passwords or long-lived service-account keys

WORKFLOW

Learn and document in Become-SWE → build and publish through swe-tickerbell → protect application logic in swe-backend.

7. Data model for the first release

Collection	Example document	Purpose
users	users/{auth0UserId}	Profile, preferences, assigned role reference
lessons	lessons/{lessonId}	Title, summary, status, publication metadata
progress	progress/{userId_lessonId}	Completion, last position, timestamps
auditLogs	auditLogs/{logId}	Actor, action, resource, result, timestamp

Do not copy passwords into Firestore. Auth0 manages authentication accounts. Firestore stores only the application information needed to operate the learning platform.

8. Security baseline

- Use Auth0 Universal Login; do not create a custom password form for the first release.
- Validate access tokens on the backend for every protected API request.
- Check resource ownership as well as role permissions.
- Use a dedicated Cloud Run service account with least-privilege IAM roles.
- Use workload identity for GitHub deployments rather than committing a service-account key.
- Store backend secrets in Secret Manager and expose only approved public configuration to the browser.
- Keep Cloud Storage buckets private; let the backend issue controlled access only after authorization.
- Apply rate limits, input validation, safe error handling, and structured logs to backend endpoints.
- Require multi-factor authentication for team and admin accounts.
- Record role changes, publishing, deletion, exports, and security-setting changes in an audit log.

Public configuration is not the same as a secret

An Auth0 SPA Client ID and Auth0 Domain are identifiers used by browser code. They are not equivalent to a Client Secret. A Client Secret, Management API token, Google service-account key, database credential, or signing key must never be placed in the public frontend repository.

9. Cost and scalability logic

The design avoids a permanently running traditional server. Firebase Hosting serves static files through a CDN, Cloud Run can scale the API according to traffic, and Firestore charges according to stored data and operations. This is suitable for a small learning platform, but serverless does not mean unlimited or automatically free.

- Start with free allowances where available, but verify current quotas before activation.
- Enable Google Cloud budgets and alerts before paid backend services are used.
- Set sensible maximum Cloud Run instances during learning.
- Monitor Firestore reads and avoid wasteful repeated queries or real-time listeners.
- Use caching for public content and aggregated analytics.
- Review service usage monthly and before every scale increase.

SCALABILITY MEANS CONTROL

A scalable architecture can grow, but a responsible architecture also limits accidental spending, permissions, and operational complexity.

10. Implementation roadmap

Phase	Goal	Proof of completion
0 — Local prototype	Review pages and content model	All pages work locally; no private data
1 — Identity	Connect Auth0 login/logout	Test learner can sign in and sign out
2 — Frontend hosting	Connect Firebase Hosting to GitHub	Pull request preview works before production
3 — Backend	Deploy FastAPI to Cloud Run	Health endpoint and one protected profile endpoint
4 — Data	Create Firestore profile and progress records	Learner reads only their permitted data
5 — Roles	Add team and admin authorization	Forbidden calls return 403, not protected data
6 — Files and audit	Protect PDFs and record admin actions	Access is controlled and traceable

11. Code package supplied with this lesson

The accompanying static code package is a design and learning prototype. It includes:

- A revised landing page that explains the architecture decision.
- A complete architecture page.
- A web version of Lesson 6 with a PDF download link.
- A learner dashboard prototype.
- A team content-management prototype.
- An admin analytics prototype.
- A responsive shared stylesheet, basic navigation script, 404 page, example Auth0 public configuration, and README.

IMPORTANT

The learner, team, and admin pages are not yet protected. They use fictional data and demonstrate layout only. Real protected data must be fetched from the Cloud Run API after backend authorization.

12. Practical exercise

9. Open the landing, architecture, learner, team, and admin prototype pages locally.
10. For every visible data item, label it public, personal, team-only, or admin-only.
11. Write the API endpoint that should supply each non-public item.
12. Write the role and ownership rule that the backend should enforce.
13. Identify the audit event required for every action that changes users, roles, content, or files.
14. Review the classification before writing authentication code.

Completion checklist

- I can explain why GitHub Pages alone cannot protect published files.
- I can explain authentication and authorization in my own words.
- I can trace one request from Firebase Hosting through Auth0 and Cloud Run to Firestore.
- I know which repositories are public and which backend repository is private.
- I know that the frontend role display is not a security boundary.
- I can state the next implementation milestone: Auth0 login/logout on the local prototype.

13. Official references

Firebase Hosting documentation: <https://firebase.google.com/docs/hosting>

Firebase Hosting GitHub integration: <https://firebase.google.com/docs/hosting/github-integration>

Firestore Hosting with Cloud Run: <https://firebase.google.com/docs/hosting/cloud-run>

Firestore security and IAM: <https://firebase.google.com/docs/firestore/security/iam>

Firestore quotas and usage: <https://firebase.google.com/docs/firestore/quotas>

Auth0 authorization for B2C applications: <https://auth0.com/docs/get-started/architecture-scenarios/business-to-consumer/authorization>

Auth0 authentication and authorization flows: <https://auth0.com/docs/get-started/authentication-and-authorization-flow>

Service capabilities, quotas, and pricing can change. Review the official documentation again during implementation.

Build the smallest secure path first.

Login → protected API → one Firestore record → one authorized result

Lesson 6 • Become SWE • Ticker Bell Kft.